

Язык Rust (Базовые знания) (Часть 2)

Занятие 4



Кафедра блокчейна

План занятия

Лекция (теория)

Ownership layer: lifetimes
(`'a`), borrow checker
Типичные ошибки компилятора
и паттерны исправления
Структурирование кода: `mod`,
`use`, `pub`, `super`

Семинар (практика)

Создаём бинарный crate
(`exe`) локально
Создаём библиотечный
crate (`package`)
Делаем модули и
подключаем их в корень
Работаем с путями:
`crate::`, `super::`

Строки: String и &str

Конкатенация 2 строк

При сложении строк, ownership одной из них переходит в новую строку. Не получится сложить две одолженных String

Пример

```
let s1 = String::from("Hello, ");  
let s2 = String::from("world!");  
let s3 = s1 + &s2;
```

```
println!("{}", s3)
```

- или так -

```
let mut s1 = String::from("foo");  
let s2 = "bar";  
s1.push_str(s2);  
println!("s2 is {s2}");
```

Строки: String и &str

Конкатенация 3+ строк

Есть несколько вариантов, как можно сложить больше 2 двух строк за один заход

Пример

```
let s1 = String::from("1");  
let s2 = String::from("2");  
let s3 = String::from("3");
```

```
let s = s1 + "-" + &s2 + "-" + &s3;
```

- или так -

```
let s1 = String::from("1");  
let s2 = String::from("2");  
let s3 = String::from("3");
```

```
let s = format!("{s1}-{s2}-{s3}");
```

Коллекции: Vec<T>

Пример

```
let mut v = Vec::new();
v.push(10);
v.push(20);

for x in &v {
    println!("{}", x);
}

v.remove(1);

if let Some(last) = v.pop() {
    println!("popped {}", last);
}
```

Коллекции: Vec<T>

Пример

```
let v = vec![1, 2, 3, 4, 5];

let third: &i32 = &v[2];
println!("The third element is {third}");

let third: Option<&i32> = v.get(2);
match third {
    Some(third) => println!("The third element is {third}"),
    None => println!("There is no third element."),
}

let does_not_exist = &v[100];
```

Коллекции: Vec<T>

Итерирование

Итерировать можно, одалживая
immutable или mutable

Пример

```
let v = vec![100, 32, 57];

for i in &v {
    println!("{i}");
}

for i in &mut v {
// for x in v.iter_mut() {
    *i += 50;
}

for i in &v {
    println!("{i}");
}
```

Коллекции: Vec<T>

Задача

Дан вектор [1, 5, 3, 5, 2, 5, 0]

Пройтись по нему и удалить все пятерки. Остальные умножить на 2

Вывести результат на экран

Что может пригодиться

```
let v = vec![100, 32, 57];  
  
v.remove(i); //удаление  
  
for i in &mut v {  
    *i -= 50;  
}
```


HashMap: хеш-таблица

ОСНОВЫ

Ключ → значение
(key/value)

entry API удобно для
инкремента/инициализации
Владение: ключи/значения
обычно перемещаются
внутри map

Пример (подсчёт частоты)

```
let mut map: HashMap<&str, i32> = HashMap::new();

map.insert("apple", 3);
map.insert("banana", 5);
map.insert("orange", 2);
println!("After insert: {:?}" , map);
```

HashMap: хеш-таблица

Иммутабельные и мутабельные ссылки

`get()` возвращает
иммутабельную ссылку

`get_mut()` позволяет
получить мутабельную
ссылку

Пример (подсчёт частоты)

```
let missing = map.get("grape");  
println!("grape: {:?}", missing);
```

```
let orange = map.get("orange");  
println!("orange: {:?}", orange);
```

```
match map.get_mut("orange") {  
    Some(x) => *x += 1,  
    None => {}  
}
```

```
println!("After get_mut: {:?}", map);
```

HashMap: хеш-таблица

Пример (подсчёт частоты)

HashMap remove

```
let removed = map.remove("banana");  
println!("return: {:?}", removed); // Some(10)  
println!("After remove: {:?}", map);
```

HashMap: хеш-таблица

Частотный словарь

Самостоятельная работа

Пример (подсчёт частоты)

```
for ch in s.chars() {  
    }  
  
    *map.entry(ch).or_insert(0) *= 2;
```

Ownership layer: зачем lifetimes?

Проблема, которую решает Rust

Ссылки (&T) не должны жить дольше данных, на которые они указывают
Компилятор должен доказать отсутствие use-after-free
Lifetimes — это «аннотации времени жизни» для ссылок (часто выводятся автоматически)

Пример проблемы (dangling reference)

```
let r: &i32;  
{  
    let x = 10;  
    r = &x; //  x умрёт в конце блока  
}  
println!("{}", r);
```

Lifetimes ('a): базовая идея

Lifetime на примере

Функция `longest` возвращает ссылку на одну из двух строк. Давайте подумаем, как она выглядит

Аннотации в сигнатуре

```
fn main() {  
    let string1 = String::from("abcd");  
    let result;  
    {  
        let string2 = "xyz";  
        result = longest(string1.as_str(), string2);  
    }  
  
    println!("The longest string is {result}");  
}
```

Lifetimes ('a): базовая идея

Lifetime на примере

Компилятор такое не скомпилирует - так как не понятен lifetime для возвращаемой ссылки - как у “x” или как у “y”?

Аннотации в сигнатуре

```
fn longest(x: &str, y: &str) -> &str {  
    if x.len() > y.len() { x } else { y }  
}
```

Lifetimes ('a): базовая идея

Lifetime на примере

Мы объединяем ссылки в “группы” по lifetime.

Lifetime такой группы является наименьшим общим lifetime

Группа lifetime обозначается как 'a, 'b и т.д.

Аннотации в сигнатуре

```
fn longest<'a>(x: &'a str, y: &'a str) -> &'a str {  
    if x.len() > y.len() { x } else { y }  
}  
  
fn main() {  
    let string1 = String::from("long string is long");  
    let result;  
    {  
        let string2 = String::from("xyz");  
        result = longest(&string1, &string2);  
        println!("The longest string is {result}");  
    }  
}
```


Lifetimes ('a): базовая идея

Что нужно запомнить

'a – это имя “срока жизни ссылки”, а не данных. Данные могут жить долго, но ссылка на них может быть разрешена только на определённом участке кода – вот этот участок и описывает lifetime.

Lifetimes – это про отношения “кто кого не короче”. Когда пишут &'a T и &'b T, компилятор проверяет, что нужная ссылка живёт достаточно долго (например, 'a не короче 'b, если это требуется).

Чаще всего lifetimes писать не нужно. Компилятор сам выводит их по правилам lifetime elision – явные 'a обычно появляются только в функциях, которые возвращают ссылку, зависящую от входной ссылки.

Аннотации в сигнатуре

```
fn first<'a>(s: &'a str) -> &'a str {  
    s  
}  
  
fn max<'a>(a: &'a str, b: &'a str) ->  
    &'a str {  
    if a.len() >= b.len() { a } else {  
    b }  
}
```

Lifetime elision: когда можно не писать 'a

3 Правила

1. По умолчанию компилятор считает, что у каждой входной ссылки свой `lifetime`. То есть если у функции два параметра `&X` и `&Y`, он мысленно даёт им разные “метки времени жизни”.
2. Если входная ссылка ровно одна – её `lifetime` автоматически становится `lifetime` результата. Поэтому функции вида `fn f(x: &T) -> &T` обычно работают без явных 'a.
3. Для методов есть особое правило: если есть `&self` (или `&mut self`), то компилятор часто считает, что возвращаемая ссылка связана именно с `self` – то есть `lifetime self` “тянется” на выход.

Без явных 'a (где можно)

```
fn len(s: &str) -> usize { s.len() }

fn first(s: &str) -> &str {
    s
}

struct Holder<'a> { s: &'a str }
impl<'a> Holder<'a> {
    fn get(&self) -> &str { self.s }
}
```

Borrow checker: правила доступа

Два золотых правила

Можно много `&T` (read-only) одновременно

Или ровно один `&mut T` (write), и никаких `&T` рядом

Цель: предотвратить data races и некорректные ссылки

Пример: `&` и `&mut` в одном scope

```
let mut s = String::from("hi");  
let a = &s;  
let b = &s;      //  ok  
// let c = &mut s; //  нельзя пока a/b используются  
println!("{}", a, b);  
let c = &mut s;  //  ok после последнего использования a/b  
c.push('!');
```

Типичная ошибка: E0716 (temporary value dropped)

Почему возникает

Вы берёте ссылку на временное значение (temporary)

Временное живёт только до конца выражения

Ссылка пытается жить дольше → компилятор запрещает

Пример

```
// ❌ ссылка на temporary  
let r = &String::from("hi");  
//           ^ temporary умрёт сразу
```

```
// ✅ решение: привязать к переменной  
let s = String::from("hi");  
let r = &s;  
println!("{}", r);
```

Типичная ошибка: «ссылка живёт дольше значения»

Как чинить

Возвращать owned-тип (String)
вместо &str

Или принимать ссылку от
вызывающего кода

Или хранить данные дольше
(перенести объявление выше)

Возвращаем owned вместо ссылки

```
fn bad() -> &str {  
    let s = String::from("hi");  
    &s // ❌ s умрёт  
}  
  
fn good() -> String {  
    let s = String::from("hi");  
    s  
}
```

Паттерн-ловушка: держим ссылку и меняем Vec

Суть проблемы

Vec может “переехать” в памяти при push. Когда места внутри массива не хватает, Vec выделяет новый буфер побольше и копирует туда элементы – поэтому адреса элементов меняются.

Поэтому нельзя одновременно держать ссылку на элемент и менять Vec. Если у вас есть `&v[0]`, а потом вы сделаете `v.push(...)`, ссылка может стать неправильной – Rust запрещает это на этапе компиляции.

```
let mut v = vec![1,2,3];
let first = &v[0];
// v.push(4); // ❌ borrow conflict
// (возможна reallocation)
println!("{}", first);
```

```
// ✅ решение:
let mut v = vec![1,2,3];
v.push(4);
let first = v[0];
```

```
//или
let mut v = vec![1,2,3];
let first = v[0];
let sum = *first + 1; //отпустили
v.push(4);
```

Ментальная модель: нельзя менять контейнер, если у вас живые ссылки внутрь.

Чек-лист: как быстро чинить borrow ошибки

Паттерны решений

Сделать borrow короче: перенеси println!/использование ссылки выше, чтобы ссылка “закончилась” раньше и после этого можно было брать &mut.

Разбить код на блоки { ... }: тогда ссылки, созданные внутри блока, “умрут” при выходе из него — и дальше компилятор разрешит мутабельный доступ.

Не держать ссылку, если можно без неё:

если тип Copy — просто скопируй значение;

если нет — делай clone() осознанно (понимая, что это копирование данных).

Изменить дизайн функции: иногда проще вернуть владельческий тип (String, Vec, T), а не ссылку; или наоборот — принимать ссылку снаружи и не создавать “долгоживущие” ссылки внутри.

Для коллекций используйте правильные инструменты:

iter_mut() — когда нужно менять элементы по одному;

split_at_mut() — когда нужны две независимые &mut на разные части;

индексирование/get() — если нужно быстро обратиться к элементу, но не удерживать ссылку долго.

Подсказка: ошибка почти всегда показывает «где взяли borrow» и «где конфликт».

Структурирование кода: `crate`, `package`, `module`

Термины

`Crate` – единица компиляции
(`binary` или `library`)

`Package` – папка с `Cargo.toml`
(может содержать несколько `crates`)

`Module` – пространство имён
внутри `crate` (`mod ...`)

Типичная структура

```
my_pkg/  
Cargo.toml  
src/  
  main.rs    # binary crate entry  
  lib.rs     # library crate entry  
  bar.rs     # just a file  
  
  utils.rs   # module file  
  utils/  
    mod.rs   # module file entry  
    math.rs  
  bar/  
    helper.rs # additional files for bar.rs
```


mod и use: подключаем модули

Как работает

mod объявляет модуль и говорит компилятору, где искать файл
use импортирует путь в текущую область видимости

Пути: crate:: (корень), super:: (вверх), self:: (внутри)

Пример: модуль файлом

```
// src/main.rs
mod utils;
use crate::utils::math::add;

fn main() {
    println!("{}", add(2, 3));
}

// src/utils.rs
pub mod math;

pub fn add(a:i32,b:i32)->i32{a+b}
```

mod и use: подключаем модули

Как работает

`mod` объявляет модуль и говорит компилятору, где искать файл
`use` импортирует путь в текущую область видимости

Пути: `crate::` (корень), `super::` (вверх), `self::` (внутри)

Пример: модуль папкой

```
// src/main.rs
mod utils;
use crate::utils::math::add;

fn main() {
    println!("{}", add(2, 3));
}

// src/utils/mod.rs
pub mod math;

// src/utils/math.rs
pub fn add(a:i32,b:i32)->i32{a+b}
```

pub: видимость и API

Правило по умолчанию

Все элементы `private` по умолчанию `pub` открывает доступ снаружи модуля

`pub(crate)` — видно только внутри `crate`

`pub(super)` — видно родителю

Видимость на практике

```
mod inner {  
    pub(crate) fn internal() {}  
    pub fn public_api() {}  
    fn private_fn() {}  
}  
  
// видно: inner::public_api  
// видно в crate: inner::internal  
// не видно: inner::private_fn
```

super и пути: crate:: / super:: / self::

Когда использовать

`crate::` – абсолютный путь от корня `crate`

`super::` – обратиться к родительскому модулю

`self::` – явность внутри модуля
`use` удобно делать на уровне модуля

super:: пример

```
// src/utils/math.rs
use super::format::fmt;

pub fn
add_and_fmt(a:i32,b:i32)->String{
    fmt(a+b)
}

// src/utils/format.rs
pub fn fmt(x:i32)->String{
    format!("{}", x) }
```

cargo test

Как запускать и писать тесты в Rust

Что делает cargo test

Собирает проект в режиме тестов и запускает все `#[test]`

Юнит-тесты обычно рядом с кодом

Интеграционные тесты — в папке `tests/` (как отдельный crate)

Тест падает через `assert!/assert_eq!/panic!` — это ожидаемо

Полезные команды и атрибуты

`cargo test name_substring` — запустить часть тестов

`cargo test -- --nocapture` — показать `println!` в выводе

`#[should_panic]` — тест ожидает панику (проверяем ошибки)

Семинар: создаём проекты и модули локально

Что сделаем руками

- 1) `cargo new hello_bin` → бинарный crate (exe)
- 2) `cargo new my_lib --lib` → библиотечный crate
- 3) `cargo add my_lib --path ../my_lib`
- 4) `cargo.toml`: `"my_lib" = { path = "../my_lib" }`
- 5) В корне либы: `pub mod utils; pub fn add()...`
- 6) В приложении: `use my_lib::utils::add;`
- 7) Используем просто `add()`
- 6) Протестируем: `cargo run`

Практика 1: получаем бинарник (.exe)

Шаги

```
cargo new hello_bin  
cd hello_bin  
cargo run  
Изменяем src/main.rs  
Собираем релиз: cargo build  
--release
```

Команды

```
cargo new hello_bin  
cd hello_bin  
cargo run  
  
# release build  
cargo build --release  
  
# запуск бинарника (путь зависит от ОС)  
./target/release/hello_bin
```

Практика 2: библиотечный crate + модули

Шаги

```
cargo new my_lib --lib
```

```
В src/lib.rs: pub mod utils;
```

```
Создаём src/utils/mod.rs и  
src/utils/math.rs
```

```
Пишем pub fn и подключаем  
через use
```

```
Добавляем тесты: cargo test
```

Мини-пакет

```
# src/lib.rs
pub mod utils;

# src/utils/mod.rs
pub mod math;

# src/utils/math.rs
pub fn add(a:i32,b:i32)->i32{a+b}

# тест (в lib.rs)
#[test]
fn t(){
    assert_eq!(utils::math::add(2,3), 5); }
```


Итоги и следующий шаг

Сегодня вы должны унести

Интуицию lifetime'ов: ссылки не живут дольше данных

Borrow checker: много & или один &mut

Ошибки: E0716 / Ментальная модель / dangling ref – и как исправлять

mod/use/pub/super и навигацию по модулям

Практика: создать bin/lib crate и подключить модули